

Lab Experiment: Inheritance & Method Overriding

Implementation of Object-Oriented Polymorphism and Hierarchical Class Structures in Java

Course: Programming in Java Lab

Language: Java (JDK 8+)

Document Code: LAB-PRG-03

1. Objective

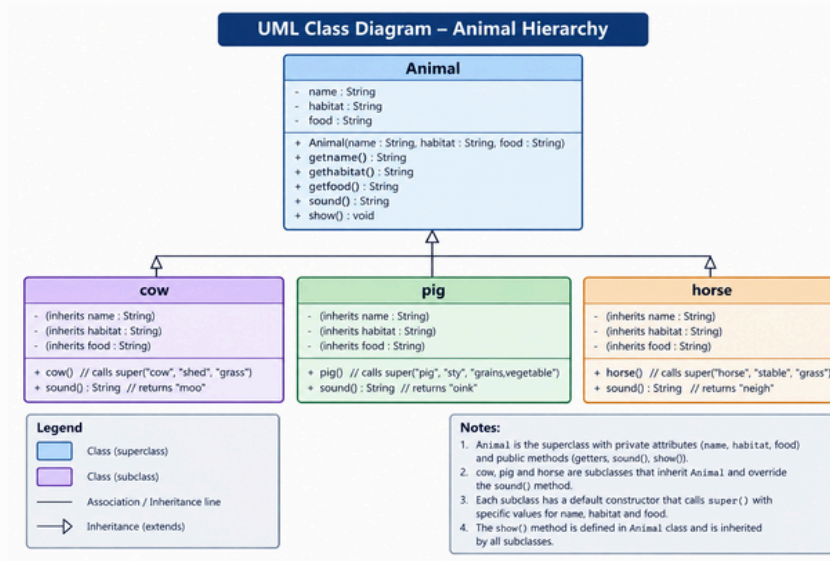
To design and implement a Java application (LabProgram3) illustrating single-level hierarchical inheritance, constructor chained initialization using the `super` keyword, runtime polymorphism through method overriding, and encapsulated data access. Key concepts include:

- **Hierarchical Inheritance:** Extending a generalized base class (Animal) into specific derived subclasses (cow, pig, horse).
- **Super Constructor Chaining:** Invoking parent class parameterised constructors using `super(...)` to set private member attributes.
- **Method Overriding:** Overriding the dynamic behavior method `sound()` in subclasses to exhibit polymorphic behavior.
- **Encapsulation:** Protecting state members with `private` modifiers while providing controlled accessors.

2. System Requirements

Component	Type	Minimum Specification	Recommended Specification
Hardware	Processor	Intel Core i3 / AMD Ryzen 3 (1.5 GHz)	Intel Core i5 / AMD Ryzen 5 (2.0 GHz+)
	RAM	1 GB	4 GB or higher
	Storage	200 MB free hard disk space	500 MB free SSD space
Software	Operating System	Windows 7 / Linux / macOS	Windows 10/11 or Ubuntu 20.04 LTS
	JDK Environment	Java Development Kit (JDK) 8	Java SE Development Kit 17 LTS or 21 LTS
	IDE / Editor	Command Line (javac / java)	Eclipse, IntelliJ IDEA, or VS Code

3. Class Diagram



4. Algorithm

Algorithm: Base Class Initialization (Animal)

1. Define private fields: name, habitat, and food.
2. In constructor `Animal(name, habitat, food)`, bind parameter values to instance variables using this references.
3. Define default polymorphic method `sound()` returning base string "animal sound".
4. Define `show()` method to output all properties, invoking dynamically overridden `sound()`.

Algorithm: Subclass Construction & Overriding

1. Create subclasses `cow`, `pig`, and `horse` extending `Animal`.
2. In subclass default constructors, execute parent constructor using `super(name, habitat, food)` with class-specific arguments.
3. Override the `sound()` method in each subclass using `@Override` to return specific vocal sounds ("moo", "oink", "neigh").

Algorithm: Main Execution Flow (LabProgram3)

1. Instantiate `horse` object named `farm`. Call `farm.show()`.
2. Instantiate `pig` object named `far`. Call `far.show()`.
3. Instantiate `cow` object named `fa`. Call `fa.show()`.

5. Compilation & Execution Instructions

Note: Save the source file with the exact name `LabProgram3.java` matching the public driver class name.

Step 1: Open Terminal / Command Prompt

```
cd path/to/your/java/files
```

Step 2: Compile the Program

```
javac LabProgram3.java
```

Step 3: Execute the Program

```
java LabProgram3
```

6. Source Code

```
class Animal {
    private String name;
    private String habitat;
    private String food;

    public Animal(String name, String habitat, String food) {
        this.name = name;
        this.habitat = habitat;
        this.food = food;
    }

    public String getname() { return name; }
    public String gethabitat() { return habitat; }
    public String getfood() { return food; }

    String sound() {
        return "animal sound";
    }

    void show() {
        System.out.println("animal is " + name);
        System.out.println("habitat is " + habitat);
        System.out.println("food is " + food);
        System.out.println("sound is " + sound());
        System.out.println("-----");
    }
}

class cowextends Animal {
    public cow() {
        super("cow", "shed", "grass");
    }

    @Override
    String sound() {
        return "moo";
    }
}

class pig extends Animal {
    public pig() {
```

```

        super("pig", "sty", "grains,vegetable");
    }

    @Override
    String sound() {
        return "oink";
    }
}

class horse extends Animal {
    public horse() {
        super("horse", "stable", "grass");
    }

    @Override
    String sound() {
        return "neigh";
    }
}

public class LabProgram3 {
    public static void main(String[] args) {
        horse farm = new horse();
        farm.show();

        pig far = new pig();
        far.show();

        cow fa = new cow();
        fa.show();
    }
}

```

7. Program Execution & Output

Terminal Execution Output

```

animal is horse
habitat is stable
food is grass
sound is neigh
-----
animal is pig
habitat is sty
food is grains,vegetable
sound is oink
-----
animal is cow
habitat is shed
food is grass
sound is moo
-----

```